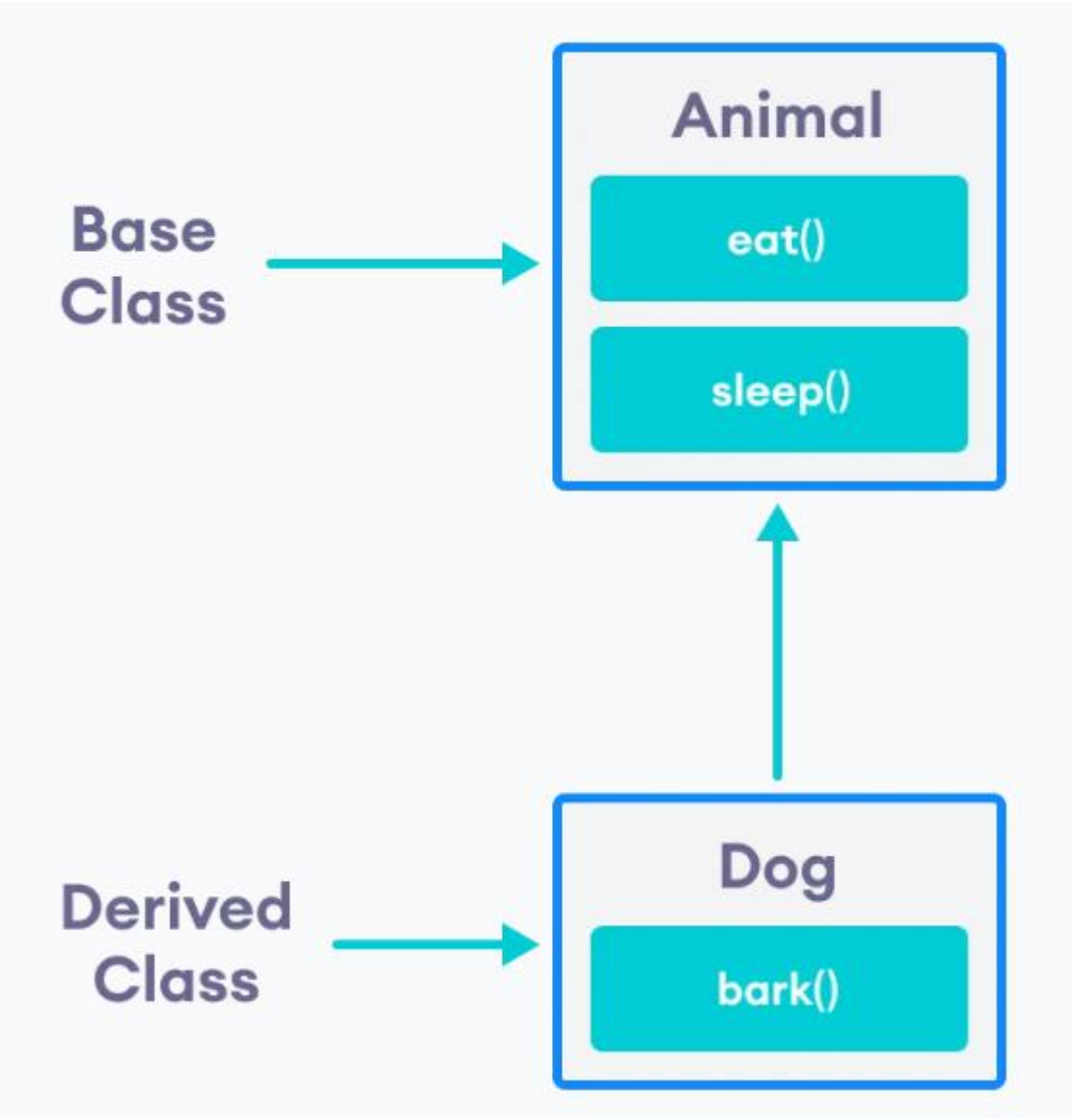
Module 4

Inheritance

- Inheritance allows us to define a class in terms of another class, which makes it easier to create and maintain an application.
- When creating a class, instead of writing completely new data members and member functions, the programmer can designate that the new class should inherit the members of an existing class.
- This existing class is called the base class, and the new class is referred to as the derived class.
- The derived classes have all the features of the base class and the programmer can choose to add new features specific to the newly created derived class.

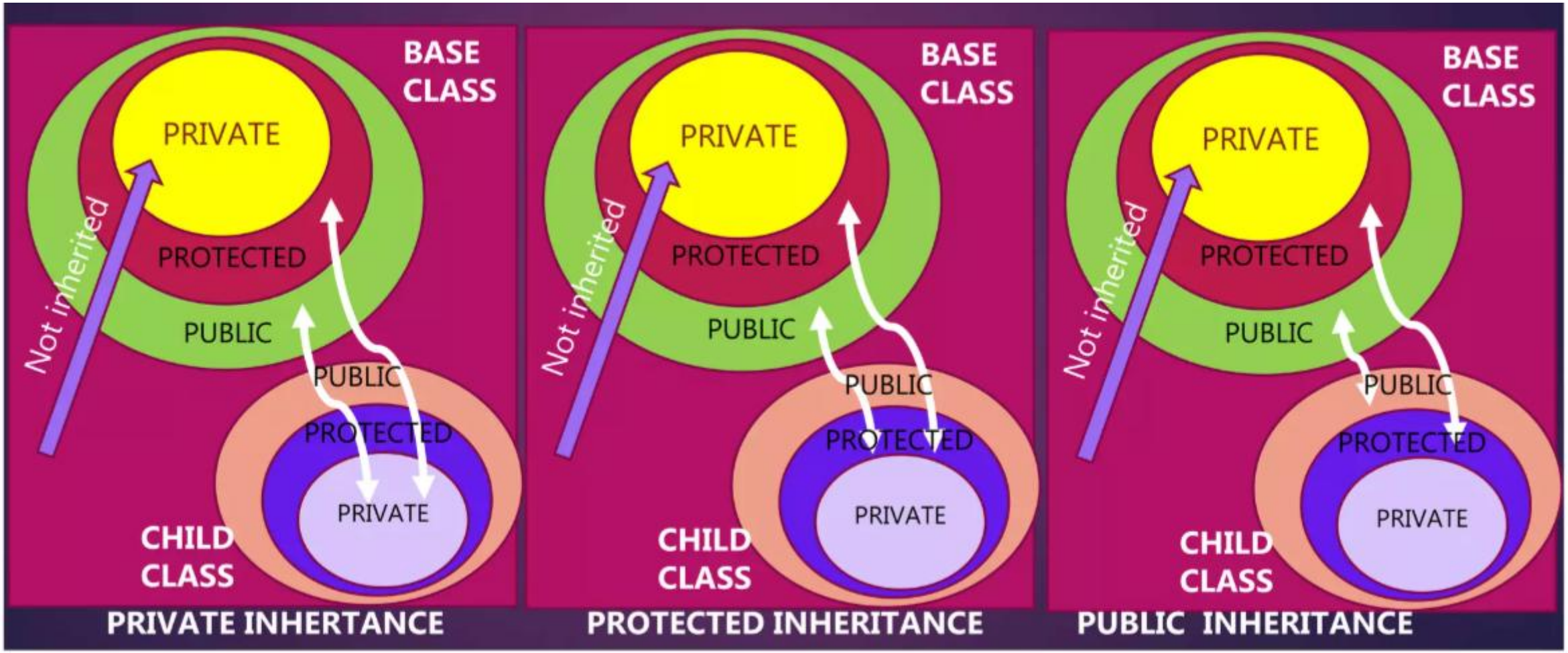

Animal
eat()
sleep()
Base Class
Dog
bark()
Derived Class

Syntax

class derived_classname : access_specifier base_classname

	Derived Class	Derived Class	Derived Class
Base Class	**Private Mode**	**Protected Mode**	**Public Mode**
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

Visibility of base class members in derived class



Private Inheritance

In private inheritance protected and public members of the base class become the private members of the derived class.

```
class base
{
private:
int a;
void funca();
protected:
int b;
void funcb();
public:
int c;
void funcc();
}
```

Private inheritance

```
class child : private base
{
private:
int x;
void funcx();
protected:
int y;
void funcy();
public:
int z;
void funcz();
}
```

```
class child
{
private:
int x;
void funcx();
int b;
void funcb();
int c;
void funcc();
protected:
int y;
void funcy();
public:
int z;
void funcz();
}
```

Protected members inherited from base class

Public members inherited from base class

New child class after inheritance

Protected Inheritance

In protected inheritance protected and public members of the base class become the protected members of the derived class.

```
class base
{
private:
int a;
void funca();
protected:
int b;
void funcb();
public:
int c;
void funcc();
}
```

Protected Inheritance

```
class child : protected base
{
private:
int x;
void funcx();
protected:
int y;
void funcy();
public:
int z;
void funcz();
}
```

```
class child
{
private:
int x;
void funcx();
protected:
int y;
void funcy();
int b;
void funcb();
int c;
void funcc();
public:
int z;
void funcz();
}
```

Protected members inherited from base class

Public members inherited from base class

New child class after inheritance

Public Inheritance

In protected inheritance protected members become the protected members of the base class and public members of the base class become the public members of the derived class.

```
class base
{
private:
int a;
void funca();
protected:
int b;
void funcb();
public:
int c;
void funcc();
}
```

Public Inheritance

```
class child : public base
{
private:
int x;
void funcx();
protected:
int y;
void funcy();
public:
int z;
void funcz();
}
```

```
class child
{
private:
int x;
void funcx();
protected:
int y;
void funcy();
int b;
void funcb();
public:
int z;
void funcz();
int c;
void funcc();
}
```

Protected members inherited from base class

Public members inherited from base class

New child class after inheritance

Types of Inheritance

- Single inheritance: A derived class inherits from a single base class. This is the simplest type of inheritance.
- Multilevel inheritance: A derived class inherits from another derived class, which in turn inherits from another derived class.
- Hierarchical inheritance: A base class is inherited by more than one derived class.
- Multiple inheritance: A class inherits properties and methods from more than one base class.
- Hybrid inheritance: A combination of multiple inheritance, single inheritance, and hierarchical inheritance.

Single Inheritance

```
#include <iostream>
using namespace std;
class Vehicle {
protected:
   string brand;
   int speed;
public:
   void setVehicle(string b, int s)    {
      brand = b;
      speed = s;
   }
   void displayVehicle()    {
      cout << "Brand: " << brand << endl;
      cout << "Speed: " << speed << endl;
   }
};
```

```
class Car : public Vehicle {
private:
   int seats;

public:
   void setCar(int s)    {
      seats = s;
   }

   void displayCar()    {
      displayVehicle();
      cout << "Number of seats: " << seats << endl;
   }
};

int main() {
   Car c;
   c.setVehicle("Toyota", 120);
   c.setCar(5);
   c.displayCar();
}
```

Multiple Inheritance

```
#include <iostream>
using namespace std;
class LandVehicle {
protected:
   int wheels;
public:
   void setWheels(int w)    {
      wheels = w;
   }
};
class WaterVehicle {
protected:
   int propellers;
public:
   void setPropellers(int p)    {
      propellers = p;
   }
};
```

```
class AmphibiousVehicle : public LandVehicle, public WaterVehicle
{
public:
   void display()
   {
      cout << "Wheels: " << wheels << endl;
      cout << "Propellers: " << propellers << endl;
   }
};

int main()
{
   AmphibiousVehicle a;
   a.setWheels(4);
   a.setPropellers(2);
   a.display();
}
```

Multilevel Inheritance

```
#include <iostream>
using namespace std;
class Vehicle {
protected:
   int speed;
public:
   void setSpeed(int s)    {
      speed = s;
   }
};

class Car : public Vehicle {
protected:
   int seats;
public:
   void setSeats(int s)    {
      seats = s;
   }
};
```

```
class SportsCar : public Car {
private:
   int turboPower;
public:
   void setTurbo(int t)    {
      turboPower = t;
   }
   void display()    {
      cout << "Speed: " << speed << endl;
      cout << "Seats: " << seats << endl;
      cout << "Turbo Power: " << turboPower << endl;
   }
};
int main() {
   SportsCar s;
   s.setSpeed(220);
   s.setSeats(2);
   s.setTurbo(500);
   s.display();
}
```

Hierarchical Inheritance

```
#include <iostream>
using namespace std;

class Vehicle  {
protected:
   string brand;
public:
   void setBrand(string b)     {
      brand = b;
   }
};

class Car : public Vehicle  {
public:
   void display()
   {
      cout << "Car Brand: " << brand << endl;
   }
};
```

```
class Bike : public Vehicle
{
public:
   void display()
   {
      cout << "Bike Brand: " << brand << endl;
   }
};

int main()
{
   Car c;
   Bike b;
   c.setBrand("Hyundai");
   b.setBrand("Yamaha");
   c.display();
   b.display();
}
```

Hybrid Inheritance

```
#include <iostream>
using namespace std;

class Vehicle
{
protected:
   string brand;
};

class Car : public Vehicle
{
protected:
   int wheels;
};

class Aircraft
{
protected:
   int wings;
};
```

```
class FlyingCar : public Car, public Aircraft
{
public:
   void setData(string b, int w, int wi)   {
      brand = b;
      wheels = w;
      wings = wi;
   }
   void display()   {
      cout << "Brand: " << brand << endl;
      cout << "Wheels: " << wheels << endl;
      cout << "Wings: " << wings << endl;
   }
};

int main(){
   FlyingCar f;
   f.setData("FutureX", 4, 2);
   f.display();
}
```

Virtual

- In hybrid inheritance, a common problem called the Diamond Problem occurs when two derived classes inherit from the same base class and another class inherits from both of them. This can create duplicate copies of the base class.
- To solve this, virtual inheritance is used so that only one copy of the base class exists.

Hybrid Inheritance

```
#include <iostream>
using namespace std;
class Vehicle {
protected:
   string brand;
public:
   void setBrand(string b)    {
      brand = b;
   }
};
class Car : virtual public Vehicle {
protected:
   int wheels;
public:
   void setWheels(int w)    {
      wheels = w;
   }
};
class Aircraft : virtual public Vehicle {
protected:
   int wings;
public:
   void setWings(int w)    {
      wings = w;
   }
};
class FlyingCar : public Car, public Aircraft {
public:
   void display()    {
      cout << "Brand: " << brand << endl;
      cout << "Number of Wheels: " << wheels << endl;
      cout << "Number of Wings: " << wings << endl;
   }
};
int main() {
   FlyingCar f;
   f.setBrand("SkyDrive");
   f.setWheels(4);
   f.setWings(2);
   f.display();
   return 0;
```

Order of Constructor/ Destructor Call

- If we inherit a class from another class and create an object of the derived class, default constructor of all of the base classes will be invoked first. Then the default constructor of the derived class will be invoked.
- In multiple inheritance, the base class's constructors are called in the order of inheritance and then the derived class constructor is called.
- Whenever the derived class's default constructor is called, the base class's default constructor is called automatically.
- To call the parameterized constructor of base class inside the parameterized constructor of sub class, we have to mention it explicitly.
- The parameterized constructor of base class cannot be called in default constructor of sub class, it should be called in the parameterized constructor of sub class.
- Destructors in C++ are called in the opposite order of that of Constructors.

```
#include <iostream>
using namespace std;

class Vehicle
{
public:
   Vehicle()
   {
      cout<<"Vehicle constructor\n";
   }
};

class Car : virtual public Vehicle
{
public:
   Car()
   {
      cout<<"Car constructor\n";
   }
};
```

```
class Aircraft : virtual public Vehicle
{
public:
   Aircraft()
   {
      cout<<"Aircraft constructor\n";
   }
};
class FlyingCar : public Car, public Aircraft
{
public:
   FlyingCar()
   {
      cout<<"FlyingCar constructor\n";
   }
};
int main()
{
   FlyingCar f;
}
```

```
Vehicle constructor
Car constructor
Aircraft constructor
FlyingCar constructor
```

```
#include <iostream>
using namespace std;

class Vehicle
{
public:
   Vehicle()
   {
      cout<<"Vehicle constructor\n";
   }
};

class Car :  public Vehicle
{
public:
   Car()
   {
      cout<<"Car constructor\n";
   }
};
```

```
class Aircraft :  public Vehicle
{
public:
   Aircraft()
   {
      cout<<"Aircraft constructor\n";
   }
};
class FlyingCar : public Car, public Aircraft
{
public:
   FlyingCar()
   {
      cout<<"FlyingCar constructor\n";
   }
};
int main()
{
   FlyingCar f;
}
```

```
Vehicle constructor
Car constructor
Vehicle constructor
Aircraft constructor
FlyingCar constructor
```

```
#include <iostream>
using namespace std;
class Vehicle  {
protected:
   string brand;
   int speed;
public:
   // Default constructor
   Vehicle()     {
      brand = "Unknown";
      speed = 0;
      cout << "Vehicle default constructor called" << endl;
   }
   void displayVehicle()
   {
      cout << "Brand: " << brand << endl;
      cout << "Speed: " << speed << endl;
   }
};
```

```
class Car : public Vehicle {
private:
   int seats;
public:
   // Default constructor
   Car()     {
      seats = 4;
      cout << "Car default constructor called" << endl;
   }
  void displayCar()
   {
      displayVehicle();
      cout << "Seats: " << seats << endl;
   }
};
int main()
{
   Car c1;
   cout << endl;
   c1.displayCar();
   return 0;
}
```

```
Vehicle default constructor called
Car default constructor called

Brand: Unknown
Speed: 0
Seats: 4
```

```
#include <iostream>
using namespace std;
class Vehicle  {
protected:
   string brand;
   int speed;
public:
void displayVehicle()
   {
      cout << "Brand: " << brand << endl;
      cout << "Speed: " << speed << endl;
   }
};
```

```
class Car : public Vehicle {
private:
   int seats;
public:
   // Default constructor
   Car()    {
      seats = 4;
      cout << "Car default constructor called" << endl;
   }
  void displayCar()
   {
      displayVehicle();
      cout << "Seats: " << seats << endl;
   }
};
int main()
{
   Car c1;
   cout << endl;
   c1.displayCar();
   return 0;
}
```

```
Car default constructor called

Brand:
Speed: 1651076199
Seats: 4
```

```
#include <iostream>
using namespace std;
class Vehicle {
protected:
    string brand;
    int speed;
public:
    // Parameterized constructor
    Vehicle(string b, int s)
    {
        brand = b;
        speed = s;
        cout << "Vehicle parameterized constructor called" << endl;
    }
    void displayVehicle()
    {
        cout << "Brand: " << brand << endl;
        cout << "Speed: " << speed << endl;
    }
};
```

```
class Car : public Vehicle {
private:
    int seats;
public:
    // Parameterized constructor
    Car(string b, int s, int se) : Vehicle(b, s)    {
        seats = se;
        cout << "Car parameterized constructor called" << endl;
    }
    void displayCar()
    {
        displayVehicle();
        cout << "Seats: " << seats << endl;
    }
};
int main() {
    Car c1("Toyota", 150, 5);
    cout << endl;
    c1.displayCar();
    return 0;
}
```

```
Vehicle parameterized cons
Car parameterized construc

Brand: Toyota
Speed: 150
Seats: 5
```

```cpp
#include <iostream>
using namespace std;
class Vehicle {
protected:
    string brand;
    int speed;
public:
    // Parameterized constructor
    Vehicle(string b, int s)
    {
        brand = b;
        speed = s;
        cout << "Vehicle param
    }
    void displayVehicle()
    {
        cout << "Brand: " << br
        cout << "Speed: " << sp
    }
};
```

```cpp
class Car : public Vehicle {
private:
    int seats;
public:
    Car()    {  // Default constructor
        seats = 4;
        cout << "Car default constructor called" << endl;
    }
    Car(string b, int s, int se) : Vehicle(b, s)    {
        seats = se;
        cout << "Car parameterized constructor called" << endl;
    }
```

Compilation failed due to following error(s).

```
main.cpp: In constructor 'Car::Car()':
main.cpp:35:5: error: no matching function for call to 'Vehicle::Vehicle()'
   35 |     {
      |     ^
main.cpp:13:5: note: candidate: 'Vehicle::Vehicle(std::string, int)'
   13 |     Vehicle(string b, int s)
      |     ^~~~~~~
main.cpp:13:5: note:   candidate expects 2 arguments, 0 provided
main.cpp:4:7: note: candidate: 'Vehicle::Vehicle(const Vehicle&)'
    4 | class Vehicle
```